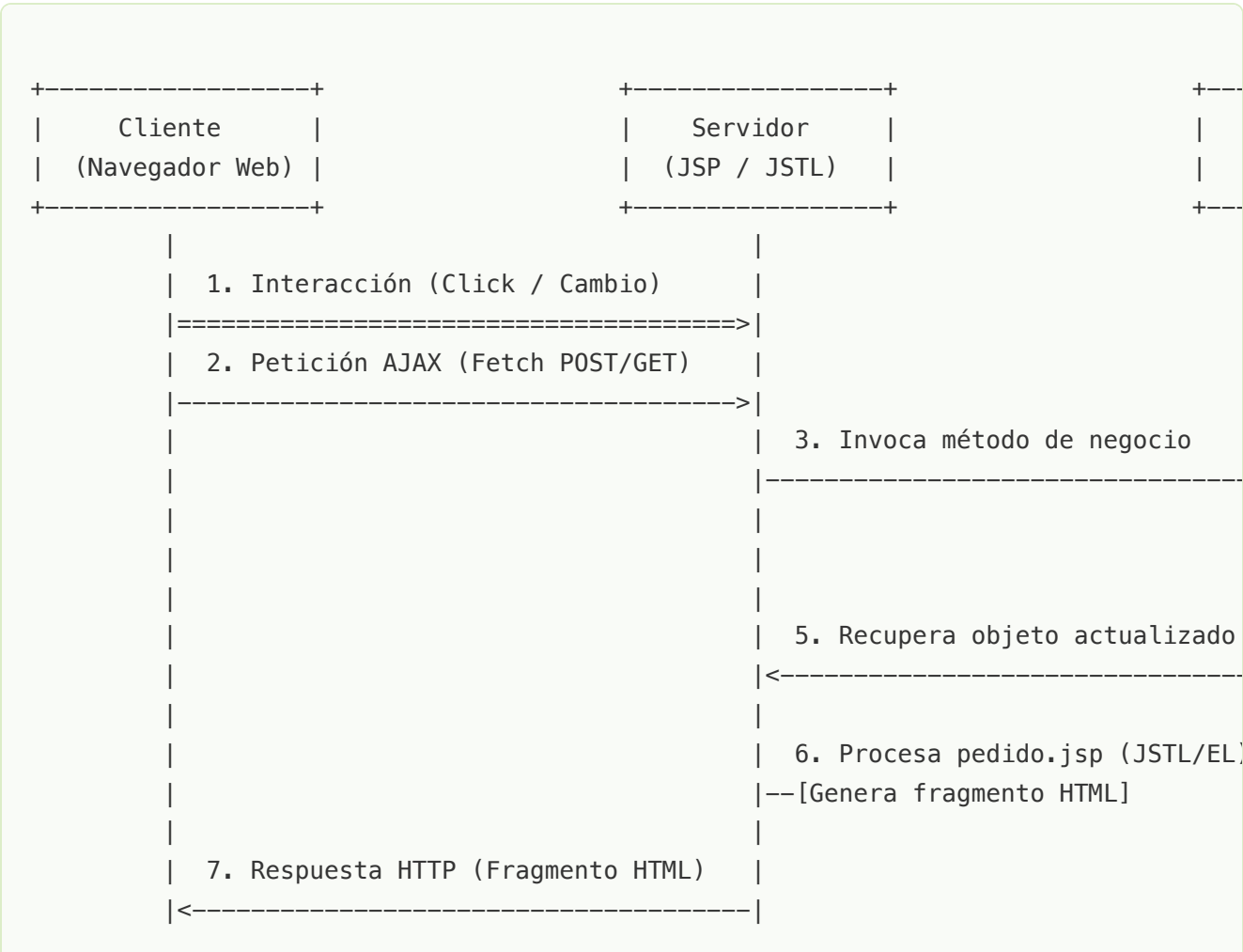


Guía de Estudio y Explicación Técnica: "Cherries"

Esta guía detalla la arquitectura, el flujo de datos y los conceptos técnicos de la administración de pedidos de la Verdulería "Cherries". Está diseñada para servir como material de estudio para tu examen parcial.

1. Arquitectura General y Flujo de Información

El proyecto implementa una arquitectura basada en **MVC (Modelo-Vista-Controlador)** desacoplada mediante **AJAX (Asynchronous JavaScript and XML)**. El intercambio de datos se realiza sin recargar la página completa, logrando una experiencia de usuario fluida y reactiva.



```
|
| 8. Reemplaza (insertAdjacentHTML)
|===[Actualiza pantalla sin recargar]
|
```

El ciclo de vida de la petición:

- **Acción del Usuario:** El usuario interactúa con la grilla (por ejemplo, marca el checkbox de entrega o quita un producto).
- **JavaScript (Controlador Cliente):**
 - El evento se propaga hasta el contenedor general (`#contenedorPedidos`) usando **Delegación de Eventos**.
 - JavaScript intercepta el evento, extrae las variables del DOM (como el ID del pedido y del producto) y realiza una petición HTTP asíncrona (`fetch`) a los JSPs de control (`check.jsp` o `quitar.jsp`).
- **Servidor JSP (Controlador Servidor):**
 - El JSP recibe los parámetros y los traduce al tipo correspondiente de forma implícita.
 - Invoca el método lógico en el **JavaBean** (`PedidoManagerBean`) para actualizar el estado.
 - Se localiza el pedido modificado en la lista y se deposita en el **Request Scope** bajo el atributo `"pedido"` .
 - Se incluye dinámicamente el fragmento `pedido.jsp` .
- **JSTL y EL (Motor de Plantillas):**
 - `pedido.jsp` lee el pedido del Request Scope y genera el código HTML correspondiente a esa tarjeta utilizando expresiones lógicas para calcular badges de estado, clases tachadas y visibilidad de botones.
 - El JSP de control devuelve el HTML resultante al navegador.
- **DOM (Actualización Visual):**
 - JavaScript recibe el HTML en la respuesta del `fetch` .
 - Busca la tarjeta afectada, inserta el nuevo HTML antes de ella usando `.insertAdjacentHTML('beforebegin', html)` y elimina la tarjeta vieja con `.remove()` (alternativa estándar y recomendada sobre `.outerHTML`).

- Si ocurre un error, el bloque `catch` revierte el estado visual del control (ej: desmarcar el checkbox si falló) y notifica al usuario en la cabecera.

2. El Lado del Cliente (El Navegador - JavaScript)

El archivo `verdu.js` actúa como el controlador del lado del cliente. Su responsabilidad es capturar interacciones, comunicarse asíncronamente y actualizar el DOM de forma quirúrgica.

A. Delegación de Eventos (Event Delegation)

En lugar de asociar un escuchador de eventos a cada checkbox y botón de la página, asociamos los eventos directamente a un ancestro común estable que nunca se destruye: el contenedor con ID `contenedorPedidos`. Esto permite que, si reemplazamos una tarjeta con nuevo HTML mediante AJAX, los nuevos elementos sigan teniendo comportamiento sin volver a configurar JavaScript.

```
document.getElementById('contenedorPedidos').addEventListener('change',  
document.getElementById('contenedorPedidos').addEventListener('click',
```

- `event.target`: Es el elemento HTML exacto donde ocurrió la acción (por ejemplo, el botón clickeado).
- `event.target.closest('.card')`: Busca hacia arriba en el árbol DOM hasta encontrar el contenedor de la tarjeta del pedido actual.

B. Estructura de la Petición AJAX (Fetch API)

Para comunicarnos con los archivos JSP, se utiliza la función moderna `fetch`.

- **Content-Type (`application/x-www-form-urlencoded`)**: Le indica al servidor que los datos viajan codificados en formato de parámetros de formulario estándar (`clave1=valor1&clave2=valor2`), lo que facilita su lectura en Java usando `request.getParameter()`.
- **Body (Cuerpo)**: Construimos la cadena dinámicamente usando plantillas de texto (Template Literals):

```
body: `id=${id}&checked=${checked}&expand=${isExpanded}`
```

C. Reemplazo del DOM y Rollback

- `card.insertAdjacentHTML('beforebegin', html)` y `card.remove()` :
Reemplaza el elemento DOM completo (la tarjeta del pedido) insertando el nuevo HTML como hermano previo y eliminando el nodo anterior. Esto logra el reemplazo de forma segura y compatible.
- **Rollback:** Si la petición falla, se ejecuta la sección `catch`, donde se restaura el checkbox a su valor lógico opuesto (`target.checked = !checked`) y se muestra el error.

3. El Lado del Servidor (JSP, JSTL y EL)

El servidor procesa la lógica de presentación utilizando **JSTL (JavaServer Pages Standard Tag Library)** y **EL (Expression Language)**, evitando estrictamente el uso de Scriptlets (`<% ... %>`).

A. Comunicación entre JSPs mediante el Request Scope

El JSP incluido (`pedido.jsp`) requiere del objeto `PedidoBean` para renderizar el HTML.

- **El Problema:** La etiqueta `<jsp:param>` solo puede transmitir cadenas de texto (Strings). Si intentamos pasar una lista de productos (`ped.productos`), JSP la convertirá a texto plano, destruyendo el objeto de tipo colección y rompiendo el bucle.
- **La Solución:** Guardar el objeto en el ámbito de la petición (**Request Scope**) usando JSTL:

```
<c:set var="pedido" value="${ped}" scope="request"/>
<jsp:include page="pedido.jsp"/>
```

Dado que el JSP incluido se ejecuta bajo el mismo flujo de petición HTTP, puede acceder al objeto original escribiendo `${pedido}` en Expression Language

conservando sus tipos de datos y colecciones Java.

B. Invocación de Métodos de JavaBean usando EL 3.0+

En versiones modernas de Expression Language, es posible invocar métodos que reciben argumentos directamente desde una expresión:

```
${listado.marcarResultrega(param.id, param.checked)}
${listado.quitarProducto(param.pedidoId, param.productoId)}
```

- **Coerción Automática:** EL convierte automáticamente el String `param.id` en `int` y el String `param.checked` en `boolean` para que coincidan con la firma del método en la clase Java.
- **Flujo de Excepciones:** Si el método lanza una excepción, el JSP fallará devolviendo un código HTTP 500, permitiendo que el JavaScript del navegador intercepte el error y dispare el rollback del checkbox.

C. Operadores Lógicos y Estructuras en JSTL

- **Control del Límite de Productos (5 inicialmente):**

```
<c:set var="limite" value="${param.expand == 'true' ? 1000 : 4}"/>
<c:forEach var="prod" items="${pedido.productos}" end="${limite}"/>
```

Establece la propiedad `end` del bucle en 1000 (para mostrar todos) si el parámetro `expand` es `true`, o en 4 (para mostrar los primeros 5 elementos, del 0 al 4) si es falso.

- **Clases Condicionales con EL:**

```
<tr class="${prod.rechazado ? 'text-danger text-decoration-line-through'}">
```

Tacha y pinta de rojo la fila del producto si se encuentra rechazado.

- **Estructura Condicional `<c:choose>` para badges de entrega:**

```
<c:if test="${pedido.listoEntrega}">
  <c:choose>
    <c:when test="${pedido.completo}">
      <span class="badge bg-success">Completo para entrega</span>
    </c:when>
    <c:otherwise>
      <span class="badge bg-warning text-dark">Parcialmente comple
    </c:otherwise>
  </c:choose>
</c:if>
```

- **Ocultamiento Condicional del Botón Quitar:**

```
class="... btn-quitar ${ (prod.rechazado || pedido.listoEntrega) ?
```

Agrega la clase `d-none` (ocultar) al botón "Quitar" si el producto ya está rechazado o si el pedido entero ya está marcado como listo para entregar.

- **Visibilidad del Botón "Ver más":**

```
<c:if test="${pedido.cantidadProductos > 5 && param.expand != 'true'}
```

Solo muestra el botón para expandir si el pedido contiene más de 5 productos y si no está actualmente expandido.

4. Persistencia ante Recarga (F5)

- **Persistencia:** Las peticiones AJAX a `check.jsp` y `quitar.jsp` llaman a los métodos del JavaBean `PedidoManagerBean`, modificando el estado de los objetos Java en el servidor (que se encuentran guardados en el ámbito `application`).
- **Visualización de 5 productos en F5:** Al presionar F5, el navegador realiza una petición GET clásica sobre `index.jsp`. Como no se le está enviando ningún parámetro `expand=true`, el valor del límite se evalúa de manera predeterminada en 4, colapsando la vista automáticamente a los primeros 5 productos.

Recomendación para el examen parcial

Recordá siempre que el uso de Scriptlets está penalizado en el criterio de aprobación. Toda estructura repetitiva o condicional debe resolverse utilizando etiquetas de JSTL. El pasaje de colecciones debe hacerse siempre a través del Request Scope (usando `c:set`) para evitar que se serialicen como String en los parámetros del include.